

Day 7 Supplementary — Big Data Internals

Hadoop · HDFS · MapReduce · Spark · Kafka · CDC · Airflow

AL-NOOR BANK EDITION

SNB DATA MANAGEMENT CAPABILITY PROGRAMME

DELIVERED BY FITCH LEARNING



One Root Cause. Seven Tools.

This is an architecture-first document. Every tool covered today is an answer to a specific version of the same problem: **a single machine hit a ceiling** — in storage, in compute, or in both — and the solution was to distribute the problem across many machines and coordinate them. Understanding which ceiling each tool addresses is more useful than memorising feature lists.

Ceiling Hit	Answer
A file is too large for one disk	HDFS — split the file across many disks, replicate for fault tolerance
A computation is too slow on one CPU	MapReduce / Spark — split the work, run in parallel, merge results
Events arrive faster than batch jobs can absorb	Kafka — durable, ordered event log that decouples producers from consumers
Pipelines need coordination, retries, and scheduling	Airflow — DAG-based orchestrator that treats pipelines as code

What We Cover Today

01

What "Big Data" Actually Means Architecturally

02

Apache Hadoop — The Cluster Model

03

HDFS — Hadoop Distributed File System

04

MapReduce — The Execution Model

05

Apache Spark — Memory over Disk

06

Apache Kafka — The Event Log

07

Change Data Capture (CDC) with Debezium

08

Apache Airflow — Pipeline Orchestration

09

How the Components Connect — Al-Noor End-to-End Pipeline



All examples use **Al-Noor Bank** as the scenario thread. SAMA regulatory context is embedded throughout.

SECTION 1

Apache Hadoop

Why it existed and how the cluster model works

Why Hadoop Was Built — The Pre-2006 Problem

Before Hadoop (pre-2006), processing a dataset that did not fit on one machine required either buying a very expensive specialised server or writing custom distributed code from scratch. Two landmark Google research papers changed the industry:

GFS Paper (2003)

Described how to store data reliably across many commodity machines — the blueprint for distributed file storage at scale.

MapReduce Paper (2004)

Described how to run computation across distributed data without moving it all to one place — the blueprint for distributed compute.

Doug Cutting and Mike Cafarella built the open-source implementation. Three problems solved simultaneously:

→ **Where do you store data too large for one machine?**

→ HDFS

→ **How do you run computation across that data without centralising it?**

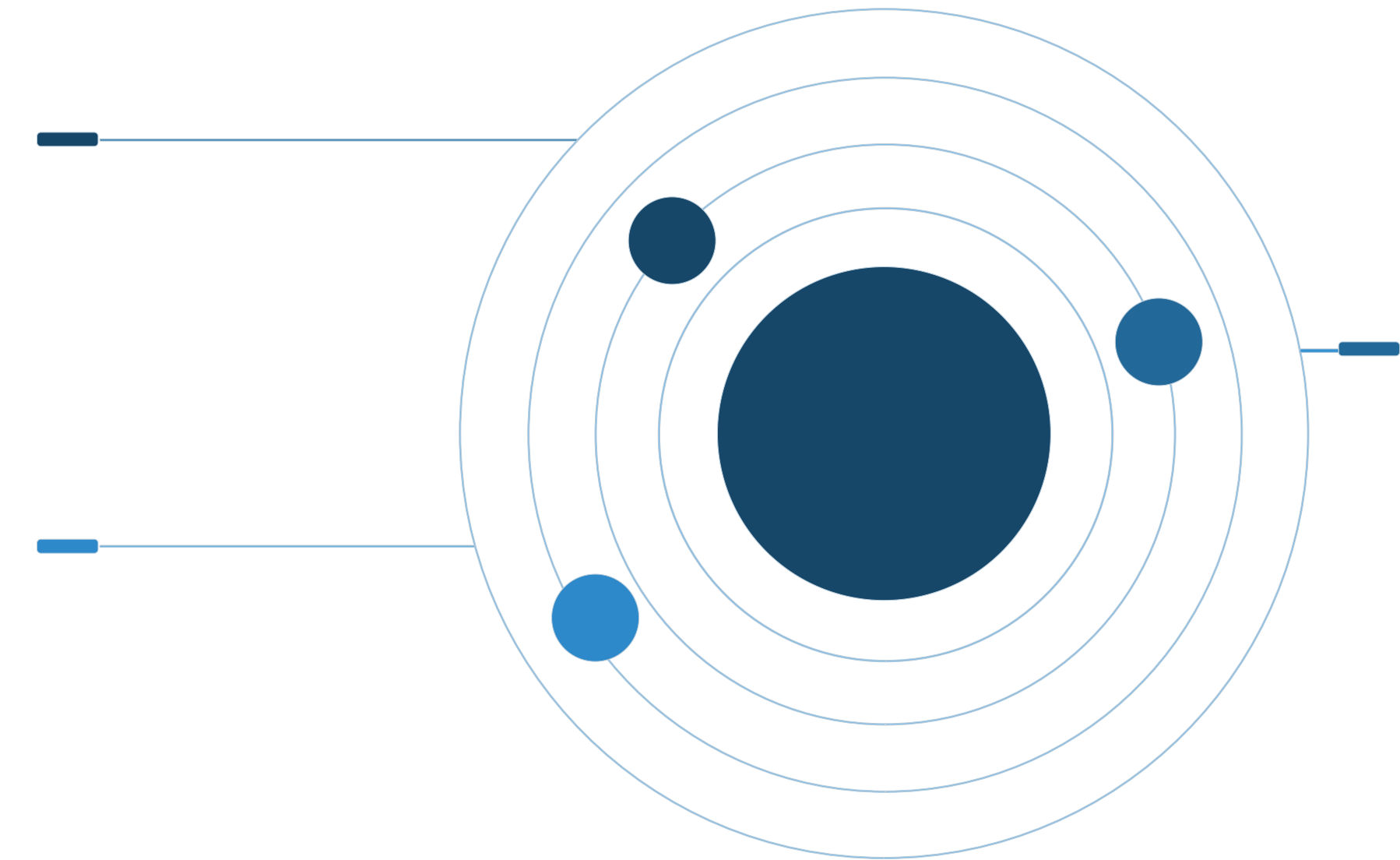
→ MapReduce

→ **How do you manage cluster resources?**

→ YARN (Yet Another Resource Negotiator)

The Cluster Model — NameNode and DataNodes

A Hadoop cluster runs on **commodity hardware** — standard servers rather than specialist storage arrays. The architectural bet: instead of buying one very reliable expensive machine, build reliability into the software layer across many cheap machines.



NameNode

The directory, not the filing cabinet. Holds the file system namespace in memory. A production cluster has a standby NameNode for high availability.

DataNodes

Store the actual data blocks. A production cluster: tens to hundreds of DataNodes, each sending heartbeats and block reports to the NameNode every few seconds.

SECTION 2

HDFS — Hadoop Distributed File System

How a file gets stored, replicated, and protected

Writing a 1 GB File to HDFS — Six Steps

Understanding the write path illuminates why HDFS is designed the way it is — the NameNode coordinates, the DataNodes do the heavy lifting, and the client drives the pipeline.

1 Client asks the NameNode where to write.

NameNode assigns DataNodes for each block. Default block size = 128 MB → 8 blocks for 1 GB.

2 Client writes block 1 to DataNode A.

DataNode A pipelines the write to DataNode B, which pipelines to DataNode C. This is the **replication pipeline**.

3 Each DataNode sends an acknowledgement back up the pipeline.

Client proceeds to block 2 only after all replicas of block 1 are confirmed.

4 Client tells the NameNode to commit file metadata once all blocks are written.

The NameNode never touches the data itself — it only updates its namespace.

NameNode metadata record for a committed file:

```
file /data/transactions/2024-01-15.csv  
= blocks [101..108], each replicated on three DataNodes
```

Replication — How HDFS Survives Hardware Failure

Replication Factor = 3

- Survives the loss of 2 DataNodes without losing any data
- NameNode detects dead DataNodes via missed heartbeats (default: 10 minutes)
- On detection: instructs surviving DataNodes to re-replicate any under-replicated blocks

Rack Awareness

- 1 replica on the local rack, 2 replicas on a remote rack
- Protects against a full rack switch failure — a single network failure cannot eliminate all copies

Al-Noor Use Case

HDFS (or its cloud equivalents — S3, ADLS) stores raw CBS extract dumps, SWIFT message archives, and historical transaction logs. Written once. Read in bulk during DWH ELT runs. Never used for anything requiring sub-second random access — that is Redis or PostgreSQL's job.

HDFS Limitations — Know the Boundaries

HDFS is purpose-built for large sequential reads and writes at scale. Knowing where it fails prevents architectural misuse.

Small Files

Each file requires a metadata entry in the NameNode's memory. One million small files consumes the same NameNode memory as one large file — the data volume is irrelevant. HDFS is optimised for thousands of large files, not millions of small ones.

Random Writes

HDFS is **append-only by design**. You cannot update a byte in the middle of a block. This eliminates the locking complexity of concurrent random writes. If you need to update individual rows, that is an operational database's job.

Low-Latency Reads

HDFS optimises for **throughput, not latency**. A single 1 GB sequential read is fast. Reading the same 100 bytes repeatedly from many small files is slow. For sub-second reads: Redis (key-value) or PostgreSQL (indexed rows).

SECTION 3

MapReduce

Parallel computation without moving data to one place

MapReduce — Split the Problem, Run in Parallel, Merge Results

MapReduce takes a problem like: *"I have 1.3 billion transaction rows spread across 200 DataNodes. I want the total transaction value per branch, per month."* It breaks this into two parallelisable phases:

Map Phase

Each worker reads its **local** data blocks and emits (key, value) pairs. No worker talks to any other worker during this phase. Computation moves to the data.

Reduce Phase

All values for the same key are collected onto one worker, which aggregates them into the final result. Workers operate independently per key group.

Al-Noor example — Map emits a branch-month key; Reduce sums all transaction values:

Al-Noor Map:

Input: txn_id=9001, branch=RUH-03, amount=15000, date=2024-01-15

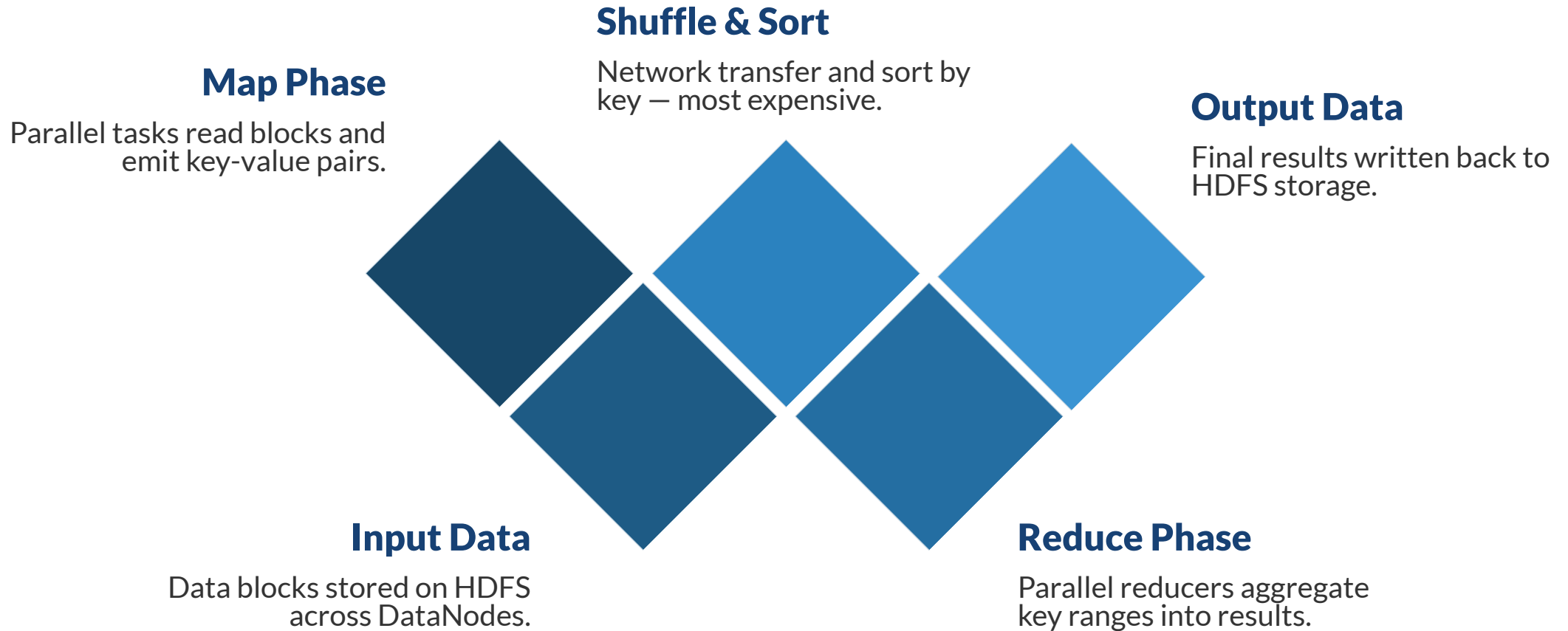
Output: ("RUH-03 | 2024-01", 15000)

Al-Noor Reduce:

Input: ("RUH-03 | 2024-01", [15000, 8200, 42000, 3100, ...])

Output: ("RUH-03 | 2024-01", 2,847,350,000)

MapReduce Execution — Five Steps



Most Expensive Phase: Shuffle and Sort — full network transfer of all intermediate data across all DataNodes. This is the architectural reason MapReduce was eventually superseded by Spark for iterative and multi-step workloads.

MapReduce's Fatal Flaw — Disk I/O at Every Step

MapReduce writes intermediate results to HDFS disk between every Map and Reduce phase. For a two-stage job this is acceptable. For an iterative algorithm — machine learning, graph traversal, multi-step SQL — you pay full disk I/O at every step.

A 10-Iteration ML Training Job in MapReduce

- 20 full HDFS read/write cycles
- Each cycle: network transfer + disk write + disk read
- Total: hours of I/O for computation that could run in minutes in memory

Why MapReduce Still Matters

MapReduce is still in production in many banks with legacy Hadoop clusters. Understanding it is necessary for reading old pipeline code, diagnosing legacy jobs, and appreciating precisely what architectural problem Spark was designed to solve.

Spark replaced this — intermediate data stays in JVM heap across the cluster. For iterative workloads: order-of-magnitude speedups.

SECTION 4

Apache Spark

Memory over disk — the architectural shift

Spark's Foundational Insight — Keep Intermediate Data in Memory

MapReduce

Intermediate data → HDFS disk → next phase reads from disk. **Every step is disk I/O.** Full network transfer between every stage.

Apache Spark

Intermediate data → JVM heap across the cluster. Next phase reads from memory. Disk is only touched when memory is insufficient (spill). **10–100× faster** for iterative workloads.

The richer API that made Spark the universal standard — beyond raw speed:



DataFrames & Spark SQL

Familiar to any analyst. SQL interface on distributed data.



Catalyst Optimizer

Automatic cost-based and rule-based query optimisation — rewrites your DAG before execution.



Delta Lake Integration

ACID transactions on data lake storage — the foundation of the AI-Noor lakehouse.

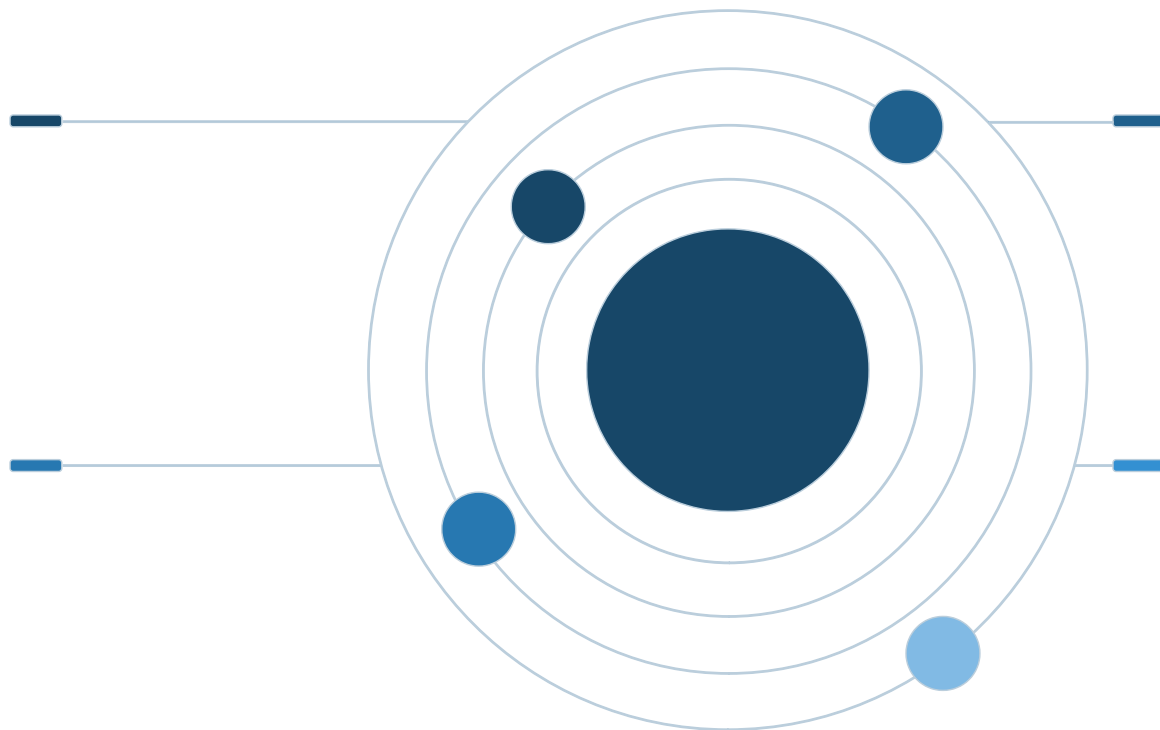


Structured Streaming

Unified batch and streaming API — one codebase for both processing modes.

Spark Architecture – Driver, Cluster Manager, Executors

Every Spark application has exactly three architectural layers. Understanding their boundaries is essential for diagnosing performance problems and sizing clusters correctly.



Driver

One per application. Runs your main program, builds the execution plan, coordinates. If Driver dies, the application fails.

Cluster Manager

Allocates CPU and memory. Does not know Spark internals — just assigns resources. YARN or Kubernetes in production.

Executors

JVM processes on worker nodes that run tasks. Fixed resources for the application lifetime. Stateless — failure triggers task retry.

Lazy Evaluation — Define First, Optimise, Then Execute

Spark collects all transformations into a **Directed Acyclic Graph (DAG)** before executing anything. The Catalyst optimizer then rewrites that DAG to eliminate unnecessary work — you may write 4 steps and Catalyst executes 2.

Two Operation Types

Transformations (lazy)

`filter()`, `map()`, `join()` — define what to do. Nothing executes until an action is called.

Actions (eager)

`count()`, `collect()`, `write()` — trigger execution of the full DAG. This is the only moment Spark does real work.

Al-Noor Example — DAG Definition

```
# Nothing executes yet — building the DAG
df = spark.read.parquet(
    "s3://al-noor-datalake/raw/transactions/")
df_filtered = df.filter(
    df.txn_date == "2024-01-15")
df_joined = df_filtered.join(
    dim_branch, "branch_id")
df_agg = df_joined.groupBy(
    "branch_name").sum("amount")

# Action triggers execution.
# Catalyst may push date filter into
# Parquet reader — skipping partitions.
df_agg.write.parquet(
    "s3://al-noor-datalake/curated/"
    "branch_daily/")
```

Partitions — The Unit of Parallelism in Spark

A Spark DataFrame is divided into **partitions**. Each partition is processed by one task on one executor core. The level of parallelism equals the number of partitions.

Too Few Partitions

Executor cores sit idle. You are paying for compute capacity that is not being used.
Throughput suffers linearly.

Optimal Range

2–4 partitions per CPU core. Each partition: 100–200 MB of data. This is the target operating range for AI-Noor ELT jobs.

Too Many Partitions

Task scheduling overhead dominates.
Thousands of tiny tasks spend more time being managed than doing work.

Two repartitioning operations and when to use each:

```
# repartition(n) — full shuffle, even distribution  
# Use when increasing partitions or fixing data skew  
df.repartition(200)
```

```
# coalesce(n) — reduces partition count WITHOUT a full shuffle  
# Use before writing output to reduce small output files  
df.coalesce(4)
```

Spark vs. MapReduce – What Changed

For the AI-Noor nightly ELT, the real advantages are the DataFrame API and Delta ACID guarantees – not raw speed alone. Understanding both systems allows engineers to diagnose legacy jobs and justify modernisation decisions.

Dimension	MapReduce	Apache Spark
Intermediate data	Written to HDFS disk	Kept in memory (spills to disk only if needed)
API	Java only (originally)	Python, Scala, Java, R, SQL
Iterative workloads	One full HDFS cycle per iteration	Cache dataset in memory, iterate without I/O
Streaming	Not native	Spark Structured Streaming
Optimiser	None	Catalyst – cost-based and rule-based
Speed vs. MapReduce (iterative)	Baseline	10–100× faster

 **AI-Noor use:** Spark runs the nightly ELT job – reads ODS transactions, joins against dimension tables in Delta Lake, writes FACT_TRANSACTION. Scheduled and orchestrated by Airflow.

SECTION 5

Apache Kafka

The distributed, durable, append-only event log

Before Kafka — Point-to-Point Integration Hell

Before event streaming platforms, service integration created a web of custom point-to-point contracts — each one a unique failure point:

The Old Pattern

System A writes to a database table. System B polls every 5 minutes. System C polls at a different interval. System D needs the data in a completely different format. Each integration is hand-crafted and fragile.

The Kafka Pattern

System A publishes an event once. Every downstream system subscribes independently and reads at its own pace. Adding a new consumer requires **zero changes to System A**.

Distributed

Spread across a cluster of broker nodes — no single point of failure.

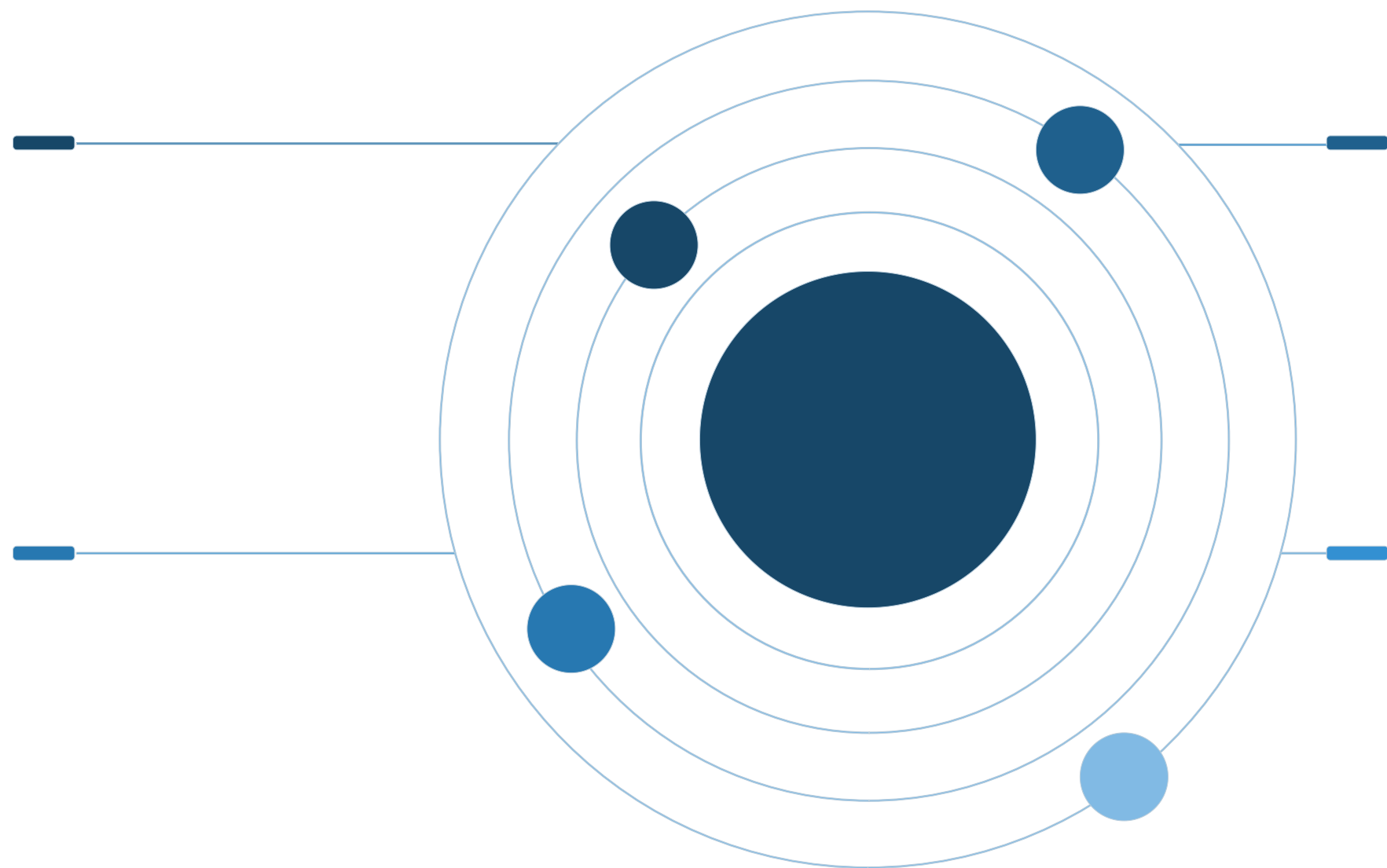
Durable

Events are written to disk and replicated before being acknowledged to the producer.

Append-Only

Events are never modified or deleted until retention expires. Every event is immutable once written.

Kafka Cluster — Brokers, Topics, Partitions



ZooKeeper vs. KRaft

Historically, Kafka relied on ZooKeeper for cluster coordination. From Kafka 2.8 onwards, **KRaft mode** replaces ZooKeeper with a built-in Raft-based metadata log.

AI-Noor Recommendation

New deployments in 2024+ should use **KRaft** — eliminates the operational complexity of running a separate ZooKeeper ensemble alongside the Kafka cluster.

Topics and Partitions — The Unit of Parallelism and Ordering

A **Topic** is a named, ordered, persistent log — think of it as a category of events: `transactions.posted` · `customers.updated` · `kyc.events`. A topic is divided into independent ordered partitions.

Topic: `transactions.posted`

Partition 0: [offset 0][offset 1][offset 2][offset 3]... → (append-only)

Partition 1: [offset 0][offset 1][offset 2]... →

Partition 2: [offset 0][offset 1]... →

...

Partition 23: [offset 0]... →

Why partition? Three independent reasons:

1 Throughput

A single partition can only be written by one thread. More partitions = higher write throughput across the cluster.

2 Distribution

Each partition is assigned to one broker as leader. Work — and storage — is spread across the cluster.

3 Consumer Parallelism

Each partition is consumed by at most one consumer per consumer group. More partitions = more parallel consumer threads.

Partition Assignment and Replication

Partition Assignment — Producer Routing

With a key (e.g., `account_id`)

`partition = hash(key) % num_partitions`. All events for the same key land on the same partition — **ordering per key is guaranteed**.

Without a key

Round-robin across all partitions. No ordering guarantees. Higher throughput, simpler routing.

i **Al-Noor:** `transactions.posted` is partitioned by `account_id`. SCD Type 2 requires ordered events per account — you cannot close the old dimension row before the new event arrives.

Replication — Fault Tolerance

Partition 0 of `transactions.posted`:

Leader: Broker 1

← producers write here

← consumers read here

Follower: Broker 2

← replicates from Broker 1

Follower: Broker 3

← replicates from Broker 1

If Broker 1 fails: controller elects a new leader from in-sync replicas (ISR). No data loss if replication was current.

Producers — Serialisation, Partitioning, and Durability Guarantees

A producer is any application that writes events to a Kafka topic. It is responsible for serialisation, partitioning, batching, and durability acknowledgement. The `acks` setting controls the durability guarantee — and for banking transactions, the choice is non-negotiable.

acks Setting	Meaning	Risk
0	Fire and forget. No acknowledgement.	Message loss on any broker failure
1	Leader acknowledges only.	Message loss if leader fails before followers replicate
all (or -1)	All in-sync replicas acknowledge.	No loss. Highest latency.



For banking transactions,

Producer Implementation — Al-Noor Transaction Event

The following producer configuration implements all SAMA DMF durability requirements: all-ISR acknowledgement, ordered retry, and Murabaha-aware event schema.

```
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers='kafka.al-noor.internal:9092',
    key_serializer=lambda k: k.encode('utf-8'),
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
    acks='all', # wait for all ISR replicas
    retries=5, # retry on transient failure
    max_in_flight_requests_per_connection=1 # preserve ordering on retry
)

producer.send(
    topic='transactions.posted',
    key='ACC-00123456', # partition key — same account → same partition
    value={
        'txn_id': 'TXN-2024-001',
        'account_id': 'ACC-00123456',
        'amount': 15000.00,
        'currency': 'SAR',
        'txn_type': 'MURABAHA_PAYMENT',
        'timestamp': '2024-01-15T14:32:00Z'
    }
)
producer.flush()
```

Consumer Groups — Parallel Processing of a Topic

A consumer reads events from one or more topic partitions, tracking its position using the **offset**. A consumer group assigns each partition to **exactly one consumer** within the group — enabling parallel processing without duplicate reads.

Topic: transactions.posted (24 partitions)

Consumer **Group**: dwh-etl-group (6 consumers)

Consumer 1 → Partitions 0, 1, 2, 3

Consumer 2 → Partitions 4, 5, 6, 7

Consumer 3 → Partitions 8, 9, 10, 11

Consumer 4 → Partitions 12, 13, 14, 15

Consumer 5 → Partitions 16, 17, 18, 19

Consumer 6 → Partitions 20, 21, 22, 23

- ✔ **Independent Consumer Groups:** A completely separate group — `aml-engine-group` — receives all the same events independently, from the beginning of retention. Adding an AML consumer requires **zero changes** to the producer or to the DWH consumer group. This is the core architectural advantage of Kafka over a traditional message queue.

Retention — Kafka Keeps Events Independent of Consumption

Kafka retains events based on configuration — **not** based on whether consumers have read them. A slow consumer falling behind does not cause messages to be deleted. Two retention policies available:

Time-Based Retention (`cleanup.policy=delete`)

Delete log segments older than N milliseconds. The right policy when every event matters but only for a defined window.

```
transactions.posted → 7 days  
customers.updated → 30 days
```

Log Compaction (`cleanup.policy=compact`)

Retain only the **most recent event per key**. Older events for the same key are deleted during background compaction. The right policy when you care about current state, not full history.

Al-Noor Retention Decisions

- `transactions.posted` → **delete** policy. 7 days in Kafka, then archived to Data Lake for 10-year SAMA retention.
- `customers.updated` → **compact** policy. You care about current customer state — not every intermediate update.

SAMA Archival

Kafka retention is the operational window. Long-term SAMA archival (10 years) is handled by the S3 sink connector writing Kafka events to the Data Lake in Parquet format, partitioned by date.

Kafka Connect — Pre-Built Connectors Without Custom Consumer Code

Running custom consumer code for every integration is operationally expensive and error-prone. Kafka Connect is a framework for running pre-built, fault-tolerant source and sink connectors — with automatic offset management.

Source Connectors

Pull data from external systems into Kafka topics. Examples: **Debezium** reading PostgreSQL WAL, JDBC polling a legacy database table.

Sink Connectors

Push data from Kafka topics into external systems. Examples: **JDBC sink** to PostgreSQL ODS, **S3 sink** to Data Lake for SAMA archival.

Connect workers are scalable, fault-tolerant, and manage offset tracking automatically — no custom consumer code required for standard integration patterns.

 **Al-Noor uses:** Debezium PostgreSQL source connector → captures CBS transaction changes into `cbs.public.transactions`. S3 sink connector → archives Kafka events to the Data Lake for 10-year SAMA retention compliance.

SECTION 6

Change Data Capture with Debezium

Why polling fails and how WAL-based CDC works

Why Polling the Database Fails for Real-Time Integration

The naive approach: query every N minutes and fetch rows where `updated_at > last_poll_time`. This fails across four independent dimensions — any one of which is disqualifying for a production banking system.

Deletes Are Invisible

A deleted row leaves no trace for a polling query to find. You will **never capture deletions** this way. In banking, account closures and transaction reversals disappear silently.

`updated_at` Is Unreliable

Not all tables have it. Some updates bypass ORM hooks. The field is maintained by the application — it can be wrong. The database engine does not guarantee it.

Production DB Load

Every poll is a read load on your production CBS. During peak hours, this competes directly with live transaction processing.

Latency Bounded by Poll Interval

A 5-minute poll means 5-minute minimum latency. **AI-Noor's AML alerts must fire within 30 seconds** of a transaction posting. Polling cannot meet this SLA.

✅ **The solution: CDC reads the database's transaction log (WAL)** instead of querying tables — no production load, no missed deletes, near-real-time latency.

PostgreSQL WAL-Based CDC — The Architecture

The Write-Ahead Log (WAL) is PostgreSQL's internal durability mechanism — every committed change is written there before it reaches the table. Debezium reads this log as a privileged replication client.



Replication Slot

A PostgreSQL object that prevents WAL entries from being discarded until Debezium has consumed them. Debezium can restart without missing changes — even across extended outages.

LSN — Log Sequence Number

A monotonically increasing WAL position. Debezium tracks the last-processed LSN — enabling exactly-once delivery semantics on reconnect.

What a Debezium Event Looks Like

Every row change emitted by Debezium carries both the **before** and **after** state of the row, the operation type, the WAL LSN, and the source table — providing a complete, auditable record of every state transition.

```
{
  "before": null,
  "after": {
    "txn_id": "TXN-2024-001",
    "account_id": "ACC-00123456",
    "amount": 15000.00,
    "txn_type": "MURABAHA_PAYMENT",
    "status": "POSTED",
    "txn_date": "2024-01-15"
  },
  "op": "c",
  "ts_ms": 1705329120000,
  "source": {
    "db": "al_noor_cbs",
    "table": "transactions",
    "lsn": 178392,
    "txId": 9001
  }
}
```

op: "c"

INSERT — new row created. **before** is null.

op: "u"

UPDATE — **before** closes the old SCD row; **after** opens the new one.

op: "d"

DELETE — **after** is null. **before** captures the deleted row state.

op: "r"

Snapshot read — initial full table capture on connector startup.

Debezium Connector — Al-Noor CBS Configuration

The following connector configuration captures changes from the CBS primary and routes them to Kafka topics prefixed with `cbs..` The `ExtractNewRecordState` transform flattens the envelope for downstream consumers that need only the current row state plus metadata.

```
{
  "name": "al-noor-cbs-connector",
  "config": {
    "connector.class":
      "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "al-noor-cbs-primary.internal",
    "database.port": "5432",
    "database.user": "debezium_reader",
    "database.password":
      "${file:/opt/secrets/dbz.properties:password}",
    "database.dbname": "al_noor_cbs",
    "plugin.name": "pgoutput",
    "slot.name": "debezium_al_noor_slot",
    "table.include.list":
      "public.transactions,public.accounts,public.customers",
    "topic.prefix": "cbs",
    "transforms": "unwrap",
    "transforms.unwrap.type":
      "io.debezium.transforms.ExtractNewRecordState",
    "transforms.unwrap.add.fields":
      "op,table,lsn,source.ts_ms",
    "heartbeat.interval.ms": "10000",
    "publication.name": "debezium_publication"
  }
}
```

CDC vs. Batch Extract — When to Use Each

The decision between CDC and batch extract is architectural — it is driven by latency requirements, source system capabilities, and regulatory completeness obligations.

Criterion	Batch Extract	CDC
Deletes captured	No	Yes
Source DB load	High (periodic scan)	Low (WAL read only)
Latency	Bounded by batch interval	Near-real-time (seconds)
Operational complexity	Low	Higher (replication slot management)
Exactly-once guarantee	Depends on updated_at reliability	Yes (LSN tracking)
SAMA auditability	Partial	Complete — every state transition captured

 **AI-Noor decision:** CDC is correct for the CBS. 500,000 transactions/day. AML alerts must fire within 30 seconds. A 5-minute batch cannot meet that SLA. Batch extract is retained only for legacy source systems that do not expose a WAL.

SECTION 7

Apache Airflow

Pipeline orchestration — DAGs, operators, scheduling, and SLAs

Why Cron and Shell Scripts Are Not Enough

The Al-Noor nightly DWH load involves eight steps in sequence — each with dependencies, failure modes, and timing constraints that cron fundamentally cannot model.

01

Wait for CBS extract to land in S3

02

Run data quality checks on the extract

03

Load the extract into the ODS

04

Run the Spark ELT job to build FACT_TRANSACTION

05

Run dbt models for dimension tables

06

Validate row counts against expected thresholds

07

Trigger SAMA report generation

08

Send completion notification

What cron cannot do: track dependencies (step 5 must not run if step 4 failed) • retry with backoff • alert on failure at 2 AM • backfill missed runs • show a UI of what ran, what failed, and how long each step took. Airflow handles all of these.

Airflow Core Concepts – DAG, Task, Operator

DAG

The pipeline definition. A Python file defining tasks and the dependencies between them. "**Acyclic**" = no loops — execution moves forward only. One Python file = one reusable, version-controlled pipeline definition.

Task

A single unit of work within a DAG. Implemented using an Operator. Has a state: `queued` · `running` · `success` · `failed` · `up_for_retry` · `skipped`.

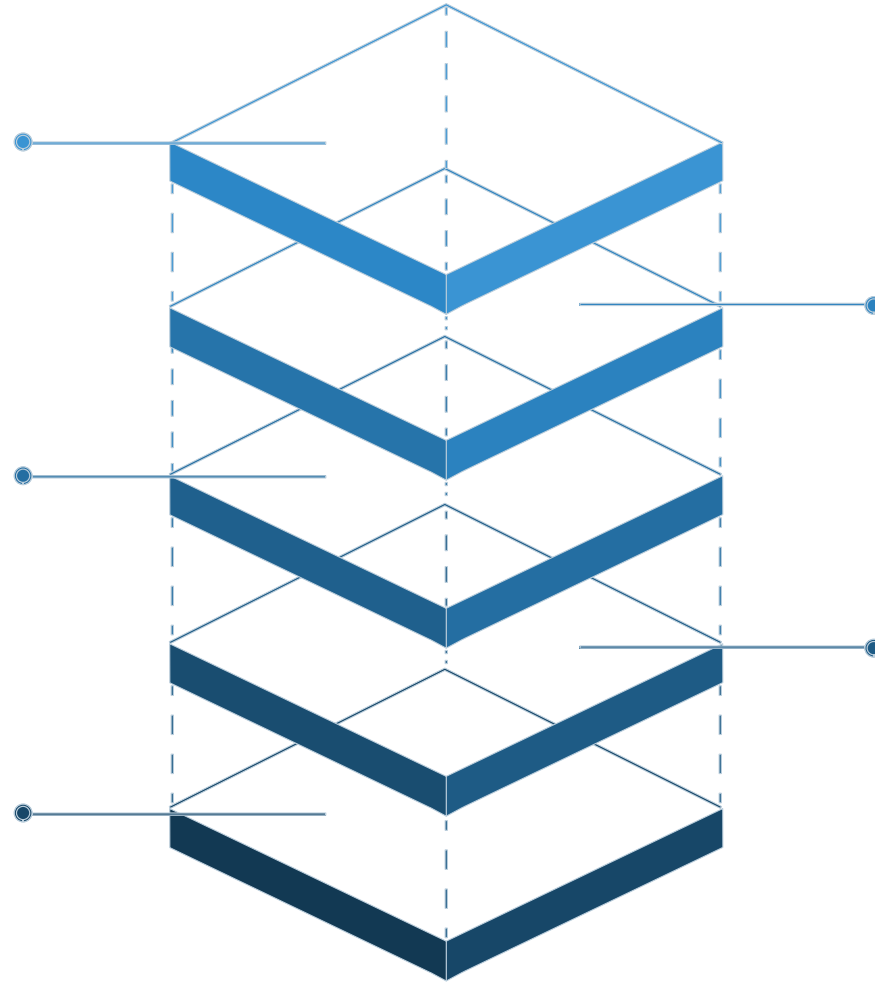
Operator

A template for a task type: `PythonOperator` · `BashOperator` · `PostgresOperator` · `SparkSubmitOperator` · `S3KeySensor` · `HttpSensor`

DAG Run & Task Instance

A **DAG Run** is one execution of the full pipeline. A **Task Instance** is one execution of one Task within one DAG Run. Both are recorded in the Metadata DB with full state history.

Airflow Architecture – Five Components



Webserver

UI and REST API. Does **not** execute tasks – display and control only.

Scheduler

Parses DAGs, checks dependencies, places ready tasks onto the work queue. Runs continuously.

Workers

Pull tasks from the queue, execute them, report results. Stateless. Horizontally scalable.

Metadata DB

PostgreSQL storing all state. Single source of truth for the cluster – every DAG run and task instance is persisted here.

Executors — How Tasks Are Run

The executor determines where and how Airflow runs tasks. Choosing the wrong executor for the environment is a common misconfiguration that causes production reliability problems.

Executor	Where Tasks Run	Use Case
SequentialExecutor	In the scheduler process, one at a time	Development only
LocalExecutor	Subprocesses on the scheduler machine	Small team, single host
CeleryExecutor	Distributed workers via message queue	Production, horizontal scale
KubernetesExecutor	Each task as a Kubernetes Pod	Cloud-native, isolation per task

 **Al-Noor production deployment:** CeleryExecutor with Redis as the broker. 4 dedicated worker nodes, each with 8 CPU cores and 32 GB RAM. Workers are stateless — scaling out is a matter of provisioning additional nodes.

Al-Noor Nightly DWH Load DAG — Setup and Configuration

The DAG definition is a Python file checked into version control. The `default_args` block establishes retry behaviour, alerting, and the SLA that protects the 07:00 SAMA reporting deadline.

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.postgres.operators.postgres import PostgresOperator
from airflow.providers.apache.spark.operators.spark_submit \
    import SparkSubmitOperator
from airflow.sensors.s3_key_sensor import S3KeySensor
from datetime import datetime, timedelta
```

```
default_args = {
    'owner': 'data-engineering',
    'retries': 2,
    'retry_delay': timedelta(minutes=5),
    'email_on_failure': True,
    'email': ['de-alerts@al-noor.sa'],
    'sla': timedelta(hours=3), # alert if DAG > 3 hours
}
```

```
with DAG(
    dag_id='al_noor_nightly_dwh_load',
    default_args=default_args,
    schedule_interval='0 1 * * *', # 01:00 AST every night
    start_date=datetime(2024, 1, 1),
    catchup=False, # do not backfill missed runs on deployment
    tags=['dwh', 'critical', 'sama'],
) as dag:
    ...
```

Al-Noor Nightly DWH Load DAG — Tasks and Dependency Chain

The `>>` operator defines the dependency chain. Each task will not start until all upstream tasks have completed successfully. Failed tasks trigger retries per `default_args`, then move to `failed` state and halt the downstream chain.

```
wait_for_extract = S3KeySensor(
    task_id='wait_for_cbs_extract',
    bucket_name='al-noor-datalake',
    bucket_key='raw/cbs/transactions/{{ ds }}/extract.parquet',
    timeout=3600, poke_interval=60,
)
validate_extract = PythonOperator(
    task_id='validate_extract',
    python_callable=validate_cbs_extract,
    op_kwargs={'date': '{{ ds }}'},
)
load_ods = PostgresOperator(
    task_id='load_ods_transactions',
    postgres_conn_id='al_noor_ods',
    sql='sql/load_ods_transactions.sql',
    parameters={'load_date': '{{ ds }}'},
)
spark_etl = SparkSubmitOperator(
    task_id='spark_fact_transaction_etl',
    application='s3://al-noor-code/spark/etl_fact_transaction.py',
    conn_id='spark_al_noor',
    application_args=['--date', '{{ ds }}'],
    conf={'spark.executor.memory': '8g',
        'spark.executor.cores': '4'},
)
validate_output = PythonOperator(
    task_id='validate_fact_output',
    python_callable=validate_fact_row_count,
    op_kwargs={'date': '{{ ds }}', 'threshold': 0.95},
)
trigger_sama = PythonOperator(
    task_id='trigger_sama_daily_report',
    python_callable=trigger_sama_pipeline,
)

# Dependency chain — left to right
wait_for_extract >> validate_extract >> load_ods \
>> spark_etl >> validate_output >> trigger_sama
```

Scheduling, Catchup, and Backfill

`schedule_interval` and `{{ ds }}`

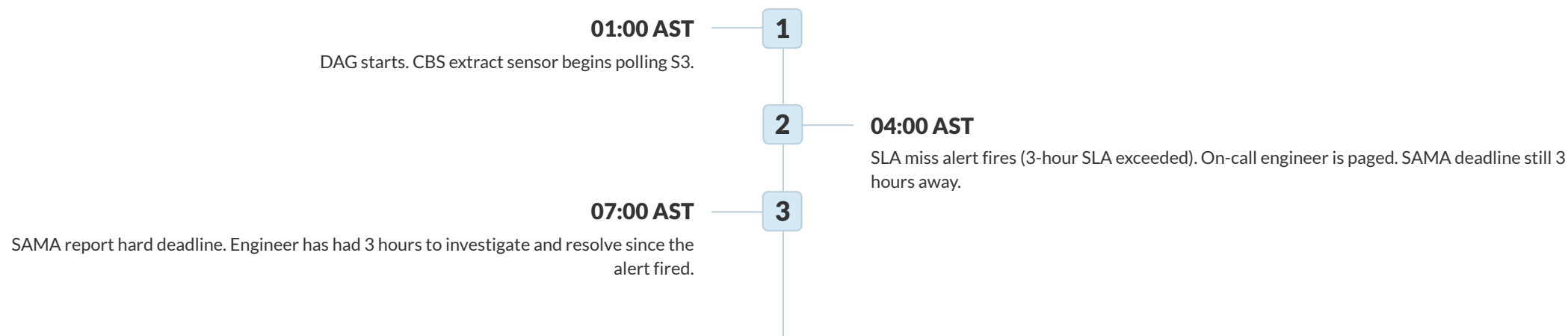
`schedule_interval` uses cron expressions or Airflow presets (`@daily`, `@hourly`). The template variable `{{ ds }}` renders to `data_interval_start` in YYYY-MM-DD format — the **date being processed**, not today's date. This is a common source of off-by-one errors in DWH pipelines.

Catchup — Handle With Care

If a DAG is paused for 10 days and re-enabled with `catchup=True`, Airflow creates 10 DAG Runs to cover missed intervals. For Al-Noor:

SLAs — The SAMA Report Deadline Problem

An SLA in Airflow is the maximum time a task or DAG run is expected to take. If exceeded, Airflow calls the `sla_miss_callback`. The task continues running — this is **not a hard stop**. It is an early-warning system.



```
def sla_miss_alert(dag, task_list, blocking_task_list,
                  slas, blocking_tis):
    send_alert(
        channel='#data-engineering-alerts',
        message=(
            f"SLA MISS: {dag.dag_id} — tasks {task_list} "
            f"exceeded SLA. SAMA report at risk. "
            f"Investigate immediately."
        )
    )
```

⚠ SLA misses in Airflow are advisory alerts — the pipeline does not stop. If the task eventually succeeds before 07:00, the SAMA report is delivered on time. The alert gives engineers the maximum possible window to intervene.

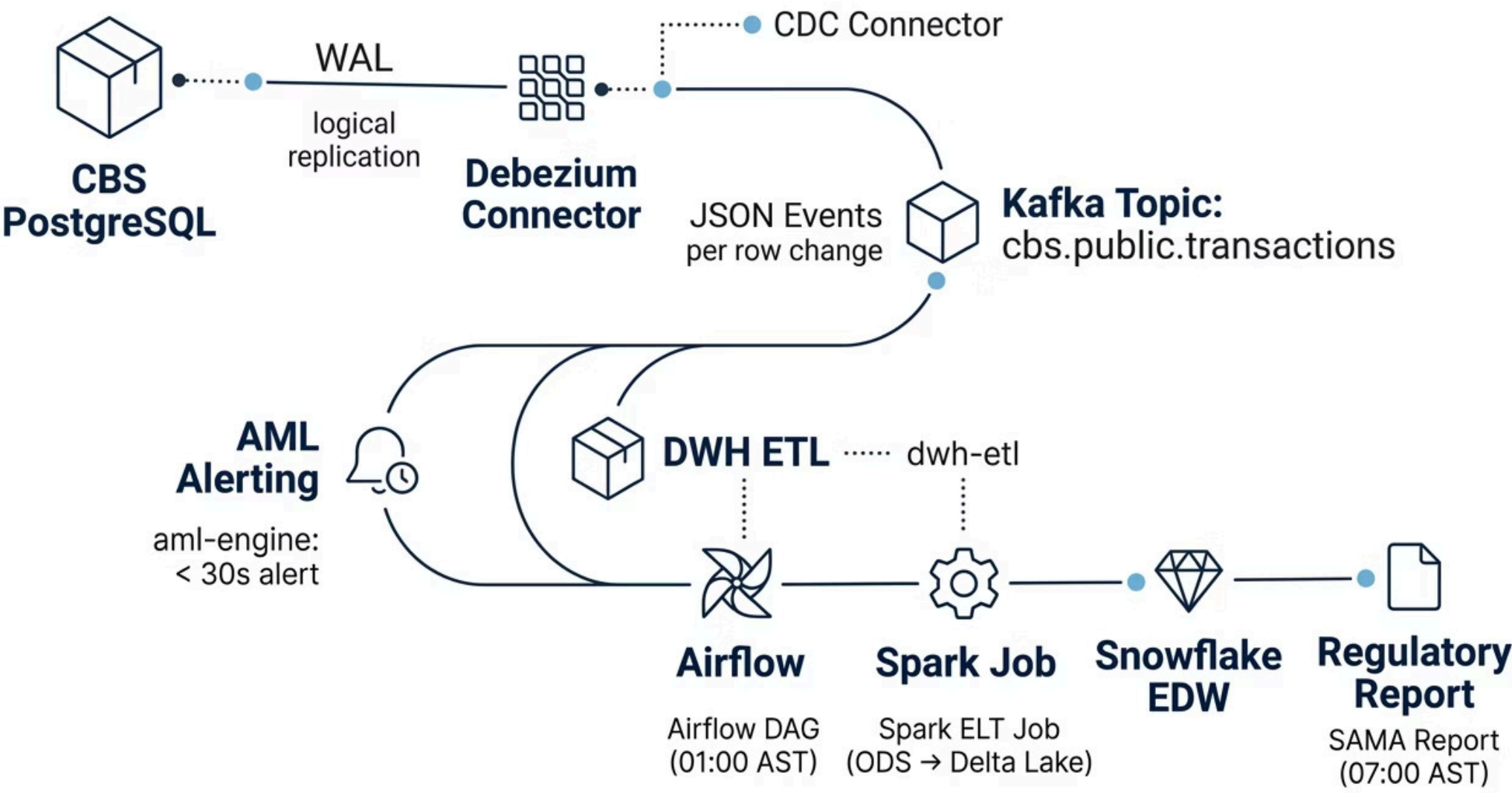
SECTION 8

How the Components Connect

The Al-Noor ingestion pipeline — end to end

Al-Noor Bank – Full Ingestion Pipeline

Two independent processing paths share a single Kafka backbone. The CDC path serves real-time AML alerting; the batch path serves daily regulatory reporting. Neither depends on the other's availability.



Key independence property: The AML engine consumes from Kafka directly for near-real-time alerting. The DWH ELT batch job reads from the ODS and produces the daily FACT_TRANSACTION load. A failure in the batch path does not affect AML alerting, and vice versa.

Key Architecture Decisions — Why Each Choice Was Made

Every architectural choice in the Al-Noor pipeline was driven by a specific technical constraint or regulatory requirement. This table is the reference for design review conversations.

Decision	Rationale
Kafka over a message queue (RabbitMQ, SQS)	Kafka retains events — consumers can replay. A message queue deletes on consumption.
CDC over batch polling	Captures deletes. No source DB scan. Sub-minute latency. SAMA-complete audit trail.
Kappa over Lambda architecture	Single processing layer. Kafka offset replay replaces a separate batch layer. Simpler operationally.
Spark over MapReduce	In-memory intermediate data. DataFrame API. Catalyst optimiser. Delta Lake integration.
Airflow over cron	Dependency tracking, retries, SLA enforcement, backfill, UI visibility. Cron does none of these.
Partition Kafka by account_id	Preserves event ordering per account. Required for correct SCD Type 2 processing.
acks=all for CBS transactions	SAMA DMF requires every transaction to be recoverable. Message loss is a regulatory finding.

Day 7 Supplementary Complete

The Snowflake EDW section — design, RBAC, Time Travel, and the Al-Noor DWH construction — is covered in the companion **DWH & Snowflake** reference document.

SNB DATA MANAGEMENT CAPABILITY PROGRAMME

DELIVERED BY FITCH LEARNING

